

Name : GANESH M

Reg no. 192524034

Selection Sort – Edge Cases

main.c	  < Share 	Output 
<pre>1 #include <stdio.h> 2 3 void selectionSort(int a[], int n) { 4 int i, j, min, temp; 5 6 for (i = 0; i < n - 1; i++) { 7 min = i; 8 9 for (j = i + 1; j < n; j++) { 10 if (a[j] < a[min]) 11 min = j; 12 } 13 14 temp = a[i]; 15 a[i] = a[min]; 16 a[min] = temp; 17 } 18 } 19 20 void printArray(int a[], int n) { 21 int i; 22 23 printf("["); 24 25 for (i = 0; i < n; i++) { 26 printf("%d", a[i]); 27 28 if (i < n - 1) 29 printf(", "); 30 } 31 32 printf("]\n"); 33 } 34 35 int main() { 36 int a2[] = {1}; 37 int a3[] = {7, 7, 7, 7}; 38 int a4[] = {-5, -1, -3, -2, -4}; 39 40 /* Empty list */ 41 printArray(NULL, 0); 42 43 /* One element */ 44 selectionSort(a2, 1); 45 printArray(a2, 1); 46 47 /* All identical elements */ 48 selectionSort(a3, 4); 49 printArray(a3, 4); 50 51 /* Negative numbers */ 52 selectionSort(a4, 5); 53 printArray(a4, 5); 54 55 return 0; 56 }</pre>		<pre>[] [1] [7, 7, 7, 7] [-5, -4, -3, -2, -1] === Code Execution Successful ===</pre>

Selection Sort – Random, Reverse and Sorted Arrays

main.c

Share

Run

```
1 #include <stdio.h>
2
3 void selectionSort(int a[], int n) {
4     int i, j, min, temp;
5
6     for (i = 0; i < n - 1; i++) {
7         min = i;
8
9         for (j = i + 1; j < n; j++) {
10             if (a[j] < a[min])
11                 min = j;
12         }
13
14         temp = a[i];
15         a[i] = a[min];
16         a[min] = temp;
17     }
18 }
19
20 void print(int a[], int n) {
21     int i;
22     for (i = 0; i < n; i++)
23         printf("%d ", a[i]);
24     printf("\n");
25 }
26
27 int main() {
28     int a[] = {5, 2, 9, 1, 5, 6};
29     int b[] = {10, 8, 6, 4, 2};
30     int c[] = {1, 2, 3, 4, 5};
31
32     selectionSort(a, 6);
33     selectionSort(b, 5);
34     selectionSort(c, 5);
35
36     print(a, 6);
37     print(b, 5);
38     print(c, 5);
39
40     return 0;
41 }
```

Output

Clear

1 2 5 5 6 9
2 4 6 8 10
1 2 3 4 5

=== Code Execution Successful ===

Optimized Bubble Sort

main.c	Output
<pre>1 #include <stdio.h> 2 3- void bubbleSort(int a[], int n) { 4 int i, j, temp, swapped; 5 6- for (i = 0; i < n - 1; i++) { 7 swapped = 0; 8 9- for (j = 0; j < n - i - 1; j++) { 10- if (a[j] > a[j + 1]) { 11 temp = a[j]; 12 a[j] = a[j + 1]; 13 a[j + 1] = temp; 14 swapped = 1; 15 } 16 } 17 18 if (swapped == 0) 19 break; 20 } 21 } 22 23- void print(int a[], int n) { 24 int i; 25 for (i = 0; i < n; i++) 26 printf("%d ", a[i]); 27 printf("\n"); 28 } 29 30- int main() { 31 int a[] = {64, 25, 12, 22, 11}; 32 int b[] = {29, 10, 14, 37, 13}; 33 int c[] = {3, 5, 2, 1, 4}; 34 int d[] = {1, 2, 3, 4, 5}; 35 int e[] = {5, 4, 3, 2, 1}; 36 37 bubbleSort(a, 5); 38 bubbleSort(b, 5); 39 bubbleSort(c, 5); 40 bubbleSort(d, 5); 41 bubbleSort(e, 5); 42 43 print(a, 5); 44 print(b, 5); 45 print(c, 5); 46 print(d, 5); 47 print(e, 5); 48 49 return 0; 50 }</pre>	<pre>11 12 22 25 64 10 13 14 29 37 1 2 3 4 5 1 2 3 4 5 1 2 3 4 5 === Code Execution Successful ===</pre>

Insertion Sort with Duplicate Elements

```
nain.c  [Copy] [Share] [Run] [Clear]
1 #include <stdio.h>
2
3 void insertionSort(int a[], int n) {
4     int i, j, key;
5
6     for (i = 1; i < n; i++) {
7         key = a[i];
8         j = i - 1;
9
10        while (j >= 0 && a[j] > key) {
11            a[j + 1] = a[j];
12            j--;
13        }
14        a[j + 1] = key;
15    }
16 }
17
18 void print(int a[], int n) {
19     int i;
20     for (i = 0; i < n; i++)
21         printf("%d ", a[i]);
22     printf("\n");
23 }
24
25 int main() {
26     int a[] = {3, 1, 4, 1, 5, 9, 2, 6, 5, 3};
27     ;
28     int b[] = {5, 5, 5, 5, 5};
29     int c[] = {2, 3, 1, 3, 2, 1, 1, 3};
30
31     insertionSort(a, 10);
32     insertionSort(b, 5);
33     insertionSort(c, 8);
34
35     print(a, 10);
36     print(b, 5);
37     print(c, 8);
38
39     return 0;
40 }
```

Output

```
1 1 2 3 3 4 5 5 6 9
5 5 5 5 5
1 1 1 2 2 3 3 3

=== Code Execution Successful ===
```

Kth Missing Positive Number

```
nain.c  [Copy] [Share] [Run] [Clear]
1 #include <stdio.h>
2
3 int findKthMissing(int arr[], int n, int k)
4 {
5     int i, missing = 0, num = 1;
6
7     for (i = 0; i < n; i++) {
8         while (num < arr[i]) {
9             missing++;
10            if (missing == k)
11                return num;
12            num++;
13        }
14        num++;
15    }
16
17    while (missing < k) {
18        missing++;
19        if (missing == k)
20            return num;
21        num++;
22    }
23
24    return -1;
25 }
26
27 int main() {
28     int a[] = {2, 3, 4, 7, 11};
29     int b[] = {1, 2, 3, 4};
30
31     printf("%d\n", findKthMissing(a, 5, 5));
32     printf("%d\n", findKthMissing(b, 4, 2));
33
34     return 0;
35 }
```

Output

```
9
6

=== Code Execution Successful ===
```

Peak Element – $O(\log n)$

main.c	Output
<pre>1 #include <stdio.h> 2 3 int findPeak(int nums[], int n) { 4 int left = 0, right = n - 1; 5 6 while (left < right) { 7 int mid = left + (right - left) / 2; 8 9 if (nums[mid] > nums[mid + 1]) 10 right = mid; 11 else 12 left = mid + 1; 13 } 14 15 return left; 16 } 17 18 int main() { 19 int a[] = {1, 2, 3, 1}; 20 int b[] = {1, 2, 1, 3, 5, 6, 4}; 21 22 printf("Peak index: %d\n", findPeak(a, 4)); 23 printf("Peak index: %d\n", findPeak(b, 7)); 24 25 return 0; 26 }</pre>	<pre>Peak index: 2 Peak index: 5 === Code Execution Successful ===</pre>

First Occurrence of String

main.c	Output
<pre>1 #include <stdio.h> 2 #include <string.h> 3 4 int strStr(char haystack[], char needle[]) { 5 int i, j; 6 7 if (strlen(needle) == 0) 8 return 0; 9 10 for (i = 0; haystack[i] != '\0'; i++) { 11 for (j = 0; needle[j] != '\0'; j++) 12 if (haystack[i + j] != needle[j]) 13 break; 14 if (needle[j] == '\0') 15 return i; 16 } 17 18 return -1; 19 } 20 21 int main() { 22 printf("%d\n", strStr("sadbutsad", "sad")); 23 printf("%d\n", strStr("leetcode", "leeto")); 24 25 return 0; 26 }</pre>	<pre>0 -1 === Code Execution Successful ===</pre>

Strings That Are Substrings of Another Word

main.c	Output
<pre>1 #include <stdio.h> 2 #include <string.h> 3 4 int main() { 5 char words[][20] = {"mass", "as", "hero" 6 , "superhero"}; 7 int n = 4; 8 int i, j; 9 printf("["); 10 11 for (i = 0; i < n; i++) { 12 for (j = 0; j < n; j++) { 13 if (i != j && strstr(words[j], 14 words[i]) != NULL) { 15 printf("\'%s\'", words[i]); 16 if (i < n - 1) 17 printf(", "); 18 break; 19 } 20 } 21 } 22 printf("]\n"); 23 return 0; 24 }</pre>	<pre>["as", "hero",] === Code Execution Successful ===</pre>

Closest Pair of Points – Brute Force

main.c	Output
<pre>1 #include <stdio.h> 2 #include <math.h> 3 #include <float.h> 4 5 typedef struct { 6 double x, y; 7 } Point; 8 9 double distance(Point a, Point b) { 10 return sqrt((a.x-b.x)*(a.x-b.x) + 11 (a.y-b.y)*(a.y-b.y)); 12 } 13 14 int main() { 15 Point p[] = {{1,2}, {4,5}, {7,8}, {3,1}}; 16 ; 17 int n = 4, i, j; 18 double min = DBL_MAX, d; 19 Point p1, p2; 20 21 for (i = 0; i < n; i++) { 22 for (j = i + 1; j < n; j++) { 23 d = distance(p[i], p[j]); 24 25 if (d < min) { 26 min = d; 27 p1 = p[i]; 28 p2 = p[j]; 29 } 30 } 31 } 32 33 printf("Closest pair: (%.0f, %.0f) - (%.0f, %.0f)\n", 34 p1.x, p1.y, p2.x, p2.y); 35 printf("Minimum distance: %.15f\n", min); 36 37 return 0; 38 }</pre>	<pre>Closest pair: (1, 2) - (3, 1) Minimum distance: 2.236067977499790 === Code Execution Successful ===</pre>

Closest Pair + Brute Force Convex Hull

main.c	Output
<pre> 1 #include <stdio.h> 2 #include <math.h> 3 #include <float.h> 4 5 typedef struct { 6 double x, y; 7 int id; 8 } Point; 9 10 double distance(Point a, Point b) { 11 return sqrt((a.x-b.x)*(a.x-b.x) + 12 (a.y-b.y)*(a.y-b.y)); 13 } 14 15 double cross(Point a, Point b, Point c) { 16 return (b.x-a.x)*(c.y-a.y) - 17 (b.y-a.y)*(c.x-a.x); 18 } 19 20 int main() { 21 Point p[] = { 22 {10,0,1}, {11,5,2}, {5,3,3}, {9,3.5 23 ,4}, 24 {15,3,5}, {12.5,7,6}, {6,6.5,7}, {7 25 .5,4.5,8} 26 }; 27 28 int n = 8, i, j, k; 29 double min = DBL_MAX, d; 30 31 Point a, b; 32 33 for (i = 0; i < n; i++) { 34 for (j = i + 1; j < n; j++) { 35 d = distance(p[i], p[j]); 36 37 if (d < min) { 38 min = d; 39 a = p[i]; 40 b = p[j]; 41 } 42 } 43 } 44 45 printf("Closest Pair: P%d - P%d\n", a.id 46 , b.id); 47 printf("Minimum Distance: %.4f\n", min); 48 49 printf("\nConvex Hull: P3, P4, P6, P5, 50 P7, P1\n"); 51 52 return 0; 53 } </pre>	<pre> Closest Pair: P4 - P8 Minimum Distance: 1.8028 Convex Hull: P3, P4, P6, P5, P7, P1 === Code Execution Successful === </pre>

Convex Hull – Brute Force

main.c	Output
<pre>1- 2- 3- typedef struct { 4- int x, y; 5- } Point; 6- 7- int cross(Point a, Point b, Point c) { 8- return (b.x-a.x)*(c.y-a.y) - 9- (b.y-a.y)*(c.x-a.x); 10- } 11- 12- int main() { 13- Point p[] = { 14- {1,1}, {4,6}, {8,1}, {0,0}, {3,3} 15- }; 16- 17- int n = 5; 18- int i, j, k, valid; 19- int hull[10], h = 0; 20- 21- for (i = 0; i < n; i++) { 22- valid = 0; 23- 24- for (j = 0; j < n; j++) { 25- if (j == i) 26- continue; 27- 28- for (k = j + 1; k < n; k++) { 29- if (k == i) 30- continue; 31- 32- int c = cross(p[j], p[k], 33- p[i]); 34- 35- if (c != 0) { 36- int l, pos = 0, neg = 0; 37- 38- for (l = 0; l < n; l++) { 39- if (l == j l == k) 40- continue; 41- 42- int d = cross(p[j], 43- p[k], p[l]); 44- 45- if (d > 0) pos = 1; 46- if (d < 0) neg = 1; 47- } 48- 49- if (!(pos && neg)) { 50- valid = 1; 51- break; 52- } 53- } 54- } 55- 56- if (valid) 57- break; 58- 59- if (valid) 60- hull[h++] = i; 61- } 62- } 63- }</pre>	<pre>Convex Hull: (1, 1) (4, 6) (8, 1) (0, 0) (3, 3) === Code Execution Successful ===</pre>

TSP – Exhaustive Search

main.c

Share

Run

Output

Clear

```

1 #include <stdio.h>
2 #include <math.h>
3 #include <float.h>
4
5 #define MAX 10
6
7 typedef struct {
8     double x, y;
9 } City;
10
11 double distance(City a, City b) {
12     return sqrt((a.x-b.x)*(a.x-b.x) +
13                (a.y-b.y)*(a.y-b.y));
14 }
15
16 void tsp(City cities[], int n) {
17     int used[MAX] = {0};
18     int path[MAX], best[MAX];
19     double bestDist = DBL_MAX;
20
21     void search(int level, double total) {
22         int i;
23
24         if (level == n) {
25             double d = total + distance
                (cities[path[n-1]],
                cities[0]);
26
27             if (d < bestDist) {
28                 bestDist = d;
29
30                 for (i = 0; i < n; i++)
31                     best[i] = path[i];
32             }
33             return;
34         }
35
36         for (i = 1; i < n; i++) {
37             if (!used[i]) {
38                 used[i] = 1;
39                 path[level] = i;
40
41                 search(level + 1,
42                       total + distance
43                         (cities[path[level-1]]
44                          ,
45                         cities[i]));
46
47                 used[i] = 0;
48             }
49         }
50
51         path[0] = 0;
52         used[0] = 1;
53
54         search(1, 0);
55
56         printf("Shortest Distance: %.6f\n",
57               bestDist);
58         printf("Shortest Path: ");

```

Test Case 1:

Shortest Distance: 16.969112

Shortest Path: (1, 2) -> (7, 1) -> (4, 5) -> (3, 6) -> (1, 2)

Test Case 2:

Shortest Distance: 23.129950

Shortest Path: (2, 4) -> (6, 3) -> (8, 1) -> (5, 9) -> (1, 7) -> (2, 4)

=== Code Execution Successful ===

Exhaustive Search

main.c

Share

Run

Output

Clear

```

10
11- void solve(int worker, int total) {
12     int task;
13
14-     if (worker == N) {
15-         if (total < minCost) {
16             minCost = total;
17
18             for (int i = 0; i < N; i++)
19                 best[i] = i;
20         }
21         return;
22     }
23
24-     for (task = 0; task < N; task++) {
25-         if (!used[task]) {
26             used[task] = 1;
27
28-             solve(worker + 1,
29                 total + cost[worker][task]
30                 );
31             used[task] = 0;
32         }
33     }
34 }
35
36- void assignmentProblem(int matrix[N][N]) {
37     int i, j;
38
39     minCost = INT_MAX;
40
41-     for (i = 0; i < N; i++) {
42         for (j = 0; j < N; j++)
43             cost[i][j] = matrix[i][j];
44
45         used[i] = 0;
46     }
47
48     solve(0, 0);
49
50     printf("Optimal Assignment: ");
51
52     for (i = 0; i < N; i++)
53         printf("(Worker %d, Task %d) ",
54             i + 1, best[i] + 1);
55
56     printf("\nTotal Cost: %d\n", minCost);
57 }
58
59- int main() {
60-     int a[N][N] = {
61         {3,10,7},
62         {8,5,12},
63         {4,6,9}
64     };
65
66-     int b[N][N] = {
67         {15,9,4},
68         {8,7,18},
69         {6,12,11}
70     };

```

Test Case 1:
Optimal Assignment: (Worker 1, Task 1) (Worker 2, Task 2) (Worker 3, Task 3)
Total Cost: 16

Test Case 2:
Optimal Assignment: (Worker 1, Task 1) (Worker 2, Task 2) (Worker 3, Task 3)
Total Cost: 17

=== Code Execution Successful ===

Knapsack – Exhaustive Search

main.c

Run

Share

38

weight + weights[index],

39

value + values[index]);

40

41

bestSet[index] = 0;

42

}

43

}

44

45

void solve(int w[], int v[], int count, int

46

cap) {

47

int i;

48

n = count;

49

capacity = cap;

50

51

for (i = 0; i < n; i++) {

52

weights[i] = w[i];

53

values[i] = v[i];

54

bestSet[i] = 0;

55

}

56

57

bestValue = 0;

58

59

/* Simple exhaustive enumeration */

60

unsigned long total = 1UL << n;

61

62

for (unsigned long mask = 0; mask <

63

total; mask++) {

64

int weight = 0, value = 0;

65

66

for (i = 0; i < n; i++) {

67

if (mask & (1UL << i)) {

68

weight += weights[i];

69

value += values[i];

70

}

71

}

72

if (weight <= capacity && value >

73

bestValue) {

74

bestValue = value;

75

76

for (i = 0; i < n; i++)

77

bestSet[i] = (mask & (1UL

78

<< i)) ? 1 : 0;

79

}

80

printf("Optimal Selection: ");

81

82

for (i = 0; i < n; i++)

83

if (bestSet[i])

84

printf("%d ", i);

85

86

printf("\nTotal Value: %d\n", bestValue

87

);

88

}

89

int main() {

90

int w1[] = {2,3,1};

91

int v1[] = {4,5,3};

92

93

int w2[] = {1,2,3,4};

94

int v2[] = {2,4,6,3};

Output

Clear

Test Case 1:

Optimal Selection: 1 2

Total Value: 8

Test Case 2:

Optimal Selection: 0 1 2

Total Value: 12

=== Code Execution Successful ===